Підготовка до заняття 6

Data Ingestion & Preprocessing для AI | Глосарій

ГЛОСАРІЙ · 1 / 4

Ingestion і витягування тексту

Ingestion pipeline

Шлях документа від джерела до пошукового індексу: забрати файл → витягти текст → очистити → прибрати дублікати → нарізати на чанки → додати метадані → записати. Це блок 2 курсу: на занятті 7 чанки стануть embeddings, на 8 — контекстом RAG.

→ Помилка на вході множиться далі: якщо в індекс потрапив cookie-банер чи колонтитул, RAG знайде саме його, і жоден промпт це не виправить.

Batch vs streaming

Патерн	Як працює	Коли обирати
Batch	Періодичний прогін: папка, дамп бази, експорт з Confluence раз на ніч	Документи змінюються рідко; просто й дешево
Streaming / event-driven	Подія на кожну зміну: webhook, черга, CDC (change data capture) з бази	Свіжість критична: тікети, ціни, статуси

→ Почніть з batch + інкрементальності: перезапуск обробляє лише файли, хеш яких змінився (manifest: файл → хеш), а повторний запуск з тими самими даними не дублює записи — це ідемпотентність. Streaming додає порядок подій, повтори й видалення: беріть його, коли затримка в години неприйнятна.

Витягування тексту за форматом

Формат	Що ламається	Чим
PDF з текстом	колонтитули, переноси слів, колонки, таблиці	pypdf, pdfplumber
Скан / фото	текстового шару немає взагалі	OCR: Tesseract, хмарний OCR
HTML	меню, cookie-банер, футер, скрипти	BeautifulSoup, trafilatura
DOCX	втрачаються заголовки, списки, таблиці	python-docx
TXT, CSV	кодування: cp1251 читається як «кракозябри»	charset-normalizer

→ OCR потрібен, коли на сторінці майже немає символів у текстовому шарі. Проганяти OCR на все підряд дорого, і він додає власні помилки. Для складних PDF у продакшні беруть unstructured, Docling, Apache Tika або LlamaParse.

Очищення і дедуплікація

Нормалізація

Приводимо текст до однієї канонічної форми, щоб хеш, пошук і чанкінг бачили те саме:

Проблема	Приклад	Що робимо
Unicode	nbsp, лігатура `ﬁ`, повноширинні цифри	`unicodedata.normalize("NFKC")`
Невидимі символи	zero-width space, BOM `U+FEFF`	видаляємо
Кінці рядків	CRLF з Windows	→ LF
Кодування	`Ð†Ð½ÑÑ,` замість «Інст»	визначити кодування, зберігати в UTF-8
Колонтитули	«Page 2 of 3», шапка на кожній сторінці	рядки, що повторюються на більшості сторінок
Переноси	`secur-` / `ity`	склеїти слово на межі рядка

→ Очищення не має ламати зміст: таблиці, числа, коди помилок і назви лишаються як є. Кожне правило очищення — окремий тест на реальному документі.

Дедуплікація

Тип	Як знаходимо	Приклад
Exact	**Content hash** — SHA-256 від канонічної форми тексту, а не від байтів файлу	той самий документ на спільному диску, але з CRLF і BOM
Near-duplicate	Шинґли (послідовності з 5 слів) і Jaccard ≥ 0.8; на мільйонах документів — MinHash + LSH	v1 і v2 політики, де змінено одне правило

→ Для near-dup треба вирішити, яку версію лишити: зазвичай новішу за датою. Дублі в індексі займають місця в top-k пошуку, а дві версії правила дають моделі суперечливий контекст.

Хеш файлу vs хеш вмісту

Хеш байтів відповідає на питання «чи змінився файл?» — для інкрементального запуску. Хеш нормалізованого тексту — «чи той самий зміст?» — для дедуплікації. Це два різні хеші з різними задачами.

PII і секрети

Персональні дані, паролі й токени в документах потрапляють в індекс, а звідти — у відповідь будь-кому. Маскувати їх треба на етапі ingestion, до індексу, а не в промпті.

Chunking, метадані, версіонування

Chunk і розмір чанка

Чанк — шматок тексту, який окремо потрапляє в пошук і в контекст моделі. Малий чанк втрачає контекст («це правило» — яке?), великий несе шум і коштує більше токенів. Типово 200–800 токенів; правильний розмір показує eval на заняттях 8–9, а не інтуїція.

Стратегії chunking

Стратегія	Як ріже	Ціна
Fixed-size + overlap	Вікно з N токенів, сусідні вікна перекриваються	Просто, але ріже речення, таблиці й розділи
За структурою (recursive)	Спершу заголовки, потім абзаци; таблиця не ріжеться	Зміст цілий; розміри нерівні
Semantic	Межа там, де змінюється тема (через embeddings)	Дорожче, потрібна модель
Parent-child	Шукаємо по малих чанках, у контекст віддаємо батьківський розділ	Більше записів в індексі

→ Структуру документа треба зберегти ще під час витягування: якщо заголовки DOCX перетворились на звичайний текст, різати «за розділами» вже нема за чим.

Метадані чанка

Кожен чанк несе: джерело, назву документа, шлях розділу (`Expense Policy > Limits`), дату чинності, групи доступу, хеш документа, версію пайплайна. Без них неможливо процитувати джерело, відфільтрувати застаріле чи пояснити, звідки взялась відповідь.

→ Права доступу фільтрують **під час пошуку**, до того як текст потрапить у контекст. Якщо фінансовий звіт дійшов до моделі, промпт «не показуй його» вже не захист.

Audit trail

Журнал подій пайплайна: що оброблено, що пропущено і чому (дубль, потрібен OCR, помилка). Відповідає на питання «чому цього документа немає в пошуку?» без дебагу.

Версіонування датасету

Manifest (файл → хеш) + версія пайплайна дають `dataset_version` : ті самі входи — той самий id. Зміна chunking — це нова версія індексу. Для великих даних — DVC або lakeFS.

Патерни і продакшн-практики

Шари даних (medallion)

Шар	Що там лежить	Навіщо
Raw / bronze	Оригінальні файли як є, незмінні	Можна переобробити з нуля новим пайплайном
Clean / silver	Витягнутий і нормалізований текст + метадані документа	Дедуплікація, перевірки якості
Chunks / gold	Чанки з метаданими, готові до embeddings	Індекс, який читає RAG

→ Raw ніколи не перезаписують: зміна chunking чи очищення — це новий прогін із того самого raw, а не ручна правка індексу.

Терміни, які зустрінете

Термін	Що означає
ETL vs ELT	Перетворити до запису (ETL) чи записати сирим і перетворювати потім (ELT). Для AI зазвичай ELT: raw зберігаємо, обробку можна повторити.
Connector / loader	Адаптер до джерела: Google Drive, Confluence, SharePoint, S3. Готові є в LlamaIndex, LangChain, Airbyte.
Upsert	Вставити або оновити запис за стабільним id (`source#chunk_index`), щоб повторний запуск не створював дублів.
Tombstone	Позначка «документ видалено». Без неї видалений у джерелі файл назавжди лишається в індексі.
Dead-letter queue	Куди йдуть файли, які не вдалось обробити: пайплайн не падає, а проблемні документи видно й можна повторити.
Backfill / reindex	Повний перепрогін історичних даних після зміни пайплайна або моделі embeddings.
Lineage	Ланцюжок «чанк → документ → файл → версія пайплайна»: звідки взялась кожна відповідь.
Freshness SLA	Обіцянка, за скільки зміна в джерелі з'явиться в пошуку: 5 хвилин чи до ранку. Визначає batch чи streaming.
Layout-aware parsing	Парсер, що розуміє макет сторінки: колонки, таблиці, підписи (Docling, unstructured, vision-моделі).

Contextual chunk header

До тексту чанка додають назву документа й розділу: «Expense Policy › Limits: …». Без заголовка чанк «Ліміт 120 EUR» незрозумілий ні пошуку, ні моделі: ліміт чого?

Data contract

Домовленість із власником джерела про формат: які поля, кодування, як позначаються видалення. Зміна формату без попередження тихо ламає ingestion.

ПЕРЕД ЗАНЯТТЯМ

Склонуйте репозиторій з GitHub

На занятті працюємо з кодом з репозиторію курсу. Якщо ви вже клонували його, достатньо `git pull` .

1. Репозиторій

Репозиторій: **github.com/andbilous/ai-engineeering-2**

```
git clone https://github.com/andbilous/ai-engineeering-2   # або git pull
cd ai-engineeering-2/lesson-06-data-ingestion
```

2. Бібліотеки

```
pip install -r requirements.txt
```

Ключ API і Ollama цього разу не потрібні: весь пайплайн — код, без моделі. Під час першого запуску tiktoken завантажує словник токенізатора (~1.7 МБ), тож потрібен інтернет. Python 3.10+.

Що в папці

`data/raw/`	9 документів вигаданої компанії: PDF, скан, HTML, DOCX, Markdown, TXT у cp1251
`data/access.json`	хто може читати документи з кожної папки
`data/expected.json`	що має вийти: дублі, OCR, заборонені рядки, доступи
`starter/ingest.py`	пайплайн з TODO 1–10
`starter/check.py`	перевірка результату, 14 checks

ПЕРЕД ЗАНЯТТЯМ

Домашня частина практики

Витягування й очищення робите вдома, а на занятті будуємо на ньому дедуплікацію, chunking, метадані й інкрементальний запуск. Пишіть код з AI-асистентом, але кожен рядок маєте вміти пояснити: на занятті хтось із вас покаже свій розв'язок.

Допишіть TODO 1–5 у starter/ingest.py

TODO	Що зробити
1	TXT / MD: визначити кодування замість «завжди UTF-8» (`charset-normalizer`), прибрати BOM
2	HTML: лишити статтю, прибрати меню, cookie-банер, скрипти й футер; заголовки → `#`
3	PDF: текст по сторінках через `\f` ; позначити скан без текстового шару як `needs_ocr`
4	DOCX: зберегти заголовки, нумеровані списки й таблиці (як рядки `| a | b |`)
5	`normalize()` : NFKC, невидимі символи, колонтитули й номери сторінок, переноси слів, пробіли

TODO 6–10 — на занятті. Запуск:

```
python starter/ingest.py
python starter/check.py
```

Що побачите реальний запуск

```
FOR CHAT: files 9 | indexed 8 | chunks 20 | skipped unchanged 0 | dataset
...
PASS  TODO 1: legacy cp1251 file decoded
...
FAIL  TODO 6: exact duplicate dropped
6/14 checks passed
```

Без змін у коді буде `0/14` . Після TODO 1–5 — `6/14` : перевірки 1–5 проходять, а 6–10 чекають на заняття. Порожній `dataset` теж нормально — це TODO 10.